Interneers Lab 2026

Track 1: FullStack Python/Django

Project Weekly Breakdown

Welcome to the Interneers Lab 2026! You would by now have an idea of the Interneers Lab, and what we are looking forward to for our next 10 weeks. By the end of these 10 weeks, we should have a very small, but end-to-end product for a simple “Inventory Management System”, and we will work towards building a functioning web application, powered by a backend web service!

This document contains a weekly breakdown of what we should aim to achieve every week, and every week builds upon the work of the previous week. If you finish a week with time to spare and are looking for an additional challenge, feel free to attempt the “Advanced” set of goals as well for every week! If you are not able to complete your target for a week for any reason - work, or other commitments keeping you busy, don’t worry! You can always catch up in the upcoming weeks, work with your mentors to figure out what is the best path for you to take and attempt the best as per what time permits.

Contents

Week 1

Upskilling Goal	1. Introduction to a development setup for product engineers 2. Working with a VCS repository, GitHub 3. Introduction to various development tools, environments, and their usages
Courses/Tutorials	• Basics of GitHub and Git - GitHub.com | Hello World • Python virtualenv and pip for beginners • How to write beautiful python code using PEP8 • Hexagonal development

1. Fork the starter 2026 intern preboarding repo (link)
2. Follow the README.md to make sure all required tools are installed, and verify that they are working as expected
3. Make recommended changes in the repository and verify that changes are reflecting
4. Build a simple GET API using hexagonal architecture that responds to some parameters, and test it via Postman
5. Push your changes back into the repository

Advanced

1. Based on hexagonal architecture, think about a way to layer the project and document it
2. Modify the README.md to talk about your repository, what it does, and a developer guide reflecting changes done so far (a guide on what makes a good README.md)

Week 2

Upskilling Goal	1. Introduction to Python and Django 2. Working with APIs, understanding different HTTP verbs - what they mean and when they are used

	3. Performing basic validations on input data
Courses/Tutorials	• Introduction to Python • Django Crash Course • Django - In-depth tutorial • Django - project structure • REST API Design - Best Practices

1. Create models for a Product, with basic information around name, description, category, price, brand, quantity within the warehouse, etc.
2. Author APIs for creating a new product, fetching a product, fetching a list of products, updating products (including deletion)
    a. All APIs will only perform in-memory operations
3. Add basic validations to all APIs
4. Test all APIs using an HTTP client

Advanced

1. Adding pagination to the fetch products API
2. Responding with meaningful error messages in case of errors/validation failures

Week 3

Upskilling Goal	1. Refactoring - service, and controller layer definitions 2. Adding request/response models, and database layer models 3. Basic data modeling and persistence in DB using ORMs
Courses/Tutorials	• MongoEngine tutorial (includes MongoDB intro) • MVC pattern in Django • Django - Encapsulating models and repository patterns • The Controller-Service-Repository Pattern • (Advanced Reading) Repository Pattern • (Supplementary) MongoDB with docker-compose

1. Create a ProductService layer that is used from a thin controller layer that services HTTP requests

    a. Add a repository layer that handles reading/writing from and to MongoDB
2. Setup MongoDB from python, and migrate models to use the mongoengine ORM
    a. Suggestion - Run your mongodb using docker compose and use those settings to connect your python code. This will make a for a well-contained repository that anyone can start contributing to!
3. Run a few API calls and view the collections and analyze program behavior using Compass

Advanced

1. Add audit columns created_at, updated_at to the Product model, how can you use these columns to further improve your APIs?

Week 4

Upskilling Goal	1. Building relations with data models 2. Advanced API patterns - fetching nested model data, filtering data 3. Building APIs beyond CRUD 4. Advanced validation and error handling
Courses/Tutorials	• REST API Design - Pagination, Filtering and Sorting • Django REST Framework Validators • Defining document schemas - mongoengine • (Recommended reading) MongoDB - Document Relationships

1. Create a ProductCategoryService layer, and a ProductCategory model. Include basic elements like title and description. Common examples could be - “Food”, “Kitchen Essentials”, etc.
2. Model products belonging to a category
3. Add CRUD APIs for a product category
    a. Add APIs to fetch a list of products belonging to a particular category
4. Add APIs to add/remove products from categories
5. Add validations that will no longer allow adding products without a brand - how will we handle existing products?
6. Use bulk POST api which accepts a CSV and creates products

Advanced

1. Write seed scripts to instantiate product categories on server startup

2. Augment the fetch products API to take instead a set of rich filters, including a list of categories
3. Write a migration mechanism for existing products - when no product category construct was present

Week 5

Upskilling Goal	1. Writing automated tests - unit tests and integration tests 2. Setting up development tools in your repository - coverage, testing scripts 3. Writing seed data - keeping repository developer friendly 4. Introduction to code reviews
Courses/Tutorials	• Python Unit Testing (Emphasize Mocking) • Code Review on GitHub • Python code coverage • Integration testing with Django - Tutorial ○ *NOTE: This will not work with your exact setup, and most articles won't. Work on taking the relevant parts from this tutorial and fitting them to your project!* • Django Rest Framework API Testing

1. Write unit tests for ProductService, ProductCategoryService mocking the repository layers
2. Write seed scripts and setup tests for writing integration tests
3. Write an integration test that tests the various APIs created so far end-to-end with a local testing mongodb server
4. Push test case for mentor review, and beyond this point we stop merging directly to main
    a. Beyond this all changes should go via a pull request. To avoid being blocked on their mentor's time - utilize the group to get reviews, or perform self-reviews, but they should stop direct merges to main
    b. Work with mentor (and maybe other peers) to verify code changes, respond to feedback, and get the code merged to main

Advanced

1. Write simple scripts that can be used to check against regressions (automatically run tests) for any given change
2. Write unit tests using parameterized tests
3. Review a peer's code in addition to the mentor's code review
4. Get your code reviewed by a peer

Week 6

Upskilling Goal	1. Introduction to JS/HTML/CSS 2. Consuming APIs via the frontend 3. Using debugging tools in the frontend
Courses/Tutorials	• Introduction to basic web development - HTML/CSS/JS • In-depth CSS tutorial and core concepts • Getting started with JavaScript • MDN - Using CSS transitions

1. Create a new page with basic HTML components, and use plain HTML to create a display tile that displays Product information
2. Style the Product display tile with style sheets
3. Make API calls using vanilla javascript, and view incoming data in the browser console
4. Use developer tools in your browser to explore the DOM tree (HTML components), understand the box model, and view javascript queries

Advanced

1. Use the data from the API calls to modify the Product display tile dynamically
2. Taking it one step further, can we fetch data and render more HTML for the entire Product list?
3. Add some pure CSS animations

Week 7

Upskilling Goal	1. Introduction to typescript and react 2. Understanding the web development service 3. Debugging React applications and working with developer tools
Courses/Tutorials	• React Tutorial (with TS) - Start with Lab 4 • Beginner tutorial for typescript • Your first component

1. Get familiar with typescript and React , including basic yarn commands to run a development server
2. Create basic components for displaying Product information, populating with dummy data

3. Add interactivity to a ProductList widget, such that clicking on a product element in the list expands to display product details (still using dummy data)

Advanced

1. Display the list of products using an <ProductList> and <Product> component
2. Add a header and navigation bar to the application

Week 8

Upskilling Goal	1. Consuming APIs, managing state, and handling data mutation on the frontend 2. Handling errors, loading state, and using higher-order components in React 3. Consuming routes and route parameters in a React application 4. Basic HTML-based browser navigation
Courses/Tutorials	• React.Dev | Managing state • Consuming APIs in React • Handling errors in React

1. Add functionality for a dedicated Product page, on which edits can be made to a product
2. Add interactivity for moving products across categories
3. Handle errors and display them to the user when some actions cannot be performed
4. Adding CRUD functionality over product categories is optional

Advanced

1. Add a product category page to view product categories, and clicking on a product category should take them to a product category page which displays product category information, and lists all products within that category - reuse <ProductList> or <Product> components here
2. Show loading spinners when refreshing data
3. Add further navigation - the product view page should display the category, which should link back to the category page

Week 9/10 (Optional)

NOTE *These 2 weeks are optional - wherein candidates may skip this week, and use it to catch up in case they have been lagging on their other weeks, or may choose to complete only parts of the 2 weeks.*

Upskilling Goal	1. Writing composable frontend components 2. Handling errors and surfacing remediation actions to users 3. Handling navigation across an application 4. Adding a new end-to-end feature, with product thought and documentation 5. Writing end-to-end tests
Courses/Tutorials	• react-router-dom | Linking and Navigation • Playwright Testing Intro ○ *This particular item is ideally supplemented with a webinar by a Rippling engineer* • The product minded engineer

There are no advanced criteria for these 2 weeks, as these are optional weeks. The items in these 2 weeks are open-ended and serve as a guideline on what a product engineer's work looks like. Candidates choosing to complete this week should aim at innovation and research in terms of what they would like to build, and how they would like to implement a feature given to them. If you wish to build a different feature than the one outlined here, please consult with our mentor and figure out a path for the 2 weeks and set realistic goals.

1. End-to-end feature - reporting. Some ideas for reports:
    a. Add a feature for reporting that allows reports to be generated based on the number of products in each category. Allow filtering where categories within some product count ranges are ignored
    b. Users should be able to generate a report of the number of products across pre-segmented price ranges, for each category
    c. Users should be able to generate a list of products that are running low (quantity is below some threshold), or a list of product categories where more than 10% of the products are running low
2. Add navigation controls to your application - clicking on products in a report could navigate to a dedicated product, or open it within a sub-section on the page
3. Write a small approach note on how the implementation would look like - what are the new components you will introduce, and how they will work with the existing structure
4. Write an end-to-end playwright test for any single part of the application (the flow should ideally be exhaustive - for example, a test that tests that happy path on creating a product, editing, and viewing the edits)
5. Add support for exporting CSV reports

Track 2: FullStack GoLang

Interneers Lab 2026

Track 2: FullStack GoLang

Project Weekly Breakdown

Welcome aboard! You would by now have an idea of the Interneers Lab, and what we are looking forward to for our next 10 weeks. By the end of these 10 weeks, we should have a very small, but end-to-end product for a simple **"Inventory Management System"**, and we will work towards building a functioning web application, powered by the speed and simplicity of Go!

This document contains a weekly breakdown of what we should aim to achieve every week, and every week builds upon the work of the previous week. If you finish a week with time to spare and are looking for an additional challenge, feel free to attempt the **"Advanced"** set of goals as well for every week!

If you are not able to complete your target for a week for any reason, work, or other commitments keeping you busy—don't worry! You can always catch up in the upcoming weeks, work with your mentors to figure out what is the best path for you to take and attempt the best as per what time permits.

AI tools for guided learning

Gemini Gem

NotebookLM (request access)

Contents

Week 1

Upskilling Goal	1. Introduction to a development setup for product engineers 2. Working with a VCS repository, GitHub 3. Introduction to various development tools, environments, and their usages
Courses / Title	1. GitHub.com | Hello World 2. https://go.dev/tour/basics/1 3. https://go.dev/tour/flowcontrol/1 4. https://www.geeksforgeeks.org/system-design/hexagonal-architecture-system-design/ 5. https://go.dev/blog/using-go-modules 6. https://github.com/rs/zerolog

Theme: Environment, Syntax, Hexagonal Architecture and Basic Server.

1. Setup (Days 1-2)

- **Git:** Initialize repo, learn branching/commits.
- **Docker:** Install Docker Desktop; understand images vs. containers.
- **Go:** Install latest version; run go mod init github.com/yourname/inventory-system.

2. Syntax & Basics (Day 3)

- **Topics:** Variables, Types, Control Flow (If/Else, Loops, Switch).
- **Resource:** A Tour of Go (Basics & Flow Control).

3. HTTP Server (Day 4-5)

- **Architecture**: hexagonal architecture (reference)
- **Library**: Standard net/http.

- **Endpoints:**
  - GET /hello-world: Returns "Hello World".
  - GET /hello-world?name=Interneer: Returns "Hello, Interneer".
- **Logging:** Use rs/zerolog instead of fmt.Println.
- **Testing:** Verify with Postman or Curl.

Advanced Topics:

- Based on hexagonal architecture, think about a way to layer the project and document it
- Modify the README.md to talk about your repository, what it does, and a developer guide reflecting changes done so far (a guide on what makes a good README.md)

Week 2

Upskilling Goal	1. Deep dive into Go's Data Structures (Structs, Slices, Maps) and Pointers 2. Building RESTful APIs with CRUD operations and in-memory data persistence 3. Understanding HTTP Status Codes and implementing Custom Middleware for request logging
Courses / Title	1. REST API Design - Best Practices 2. https://go.dev/doc/modules/layout 3. https://go.dev/tour/moretypes/1 4. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status **Advanced:** 1. https://github.com/golang-standards/project-layout

Theme: In-Memory Storage and API Best Practices.

1. Data Modeling (Day 1)

- **Concepts:** Pointers, Structs, Slices, Maps.
- **Model:** Create a Product struct with JSON tags.
  - *Fields:* ID, Name, Description, Category, Price, Brand, Quantity.

2. In-Memory API (Days 2-4)

- **Storage:** Use map as a temporary database.
- **Endpoints:** Implement full CRUD (POST, GET, PUT, DELETE) for /products.
- **Validation:** Price > 0, Quantity >= 0, Name required.

3. Standards & Middleware (Day 5)

- **Status Codes:** Ensure correct usage (201 Created, 204 No Content, 400 Bad Request, 404 Not Found).
- **Middleware:** Implement a wrapper to log request Method, Path, and **Execution Time** using defer.
- **Testing**: Test all API using a HTTP Client

Advanced Topics

- Understanding how tags works in struct

Week 3

Upskilling Goal	1. Refactoring - service, and controller layer definitions 2. Adding request/response models, and database layer models 3. Basic data modeling and persistence in DB using ORMs
Courses / Title	1. https://go.dev/tour/methods/1 2. repository pattern in go 3. context in go 4. https://www.mongodb.com/resources/languages/golang

Theme: Layered Architecture and MongoDB.

1. Database & Concepts (Days 1-2)

- **Learn:** Interfaces, **Context (context.Context)**, and MongoDB Driver (BSON tags).
- **Setup:** Run MongoDB via Docker

2. Refactoring: Layered Architecture (Days 3-5)

Refactor Week 2 code into strictly defined layers, ensuring **Context Propagation**:

- **A. Domain Layer:** Product struct with BSON tags.

- **B. Repository Layer (Data):**
    - Define ProductRepository interface (Create, Get, List, Update, Delete).
    - **Requirement:** All methods must accept context.Context as the first argument.
    - Implement MongoProductRepository utilizing the MongoDB driver and passing the context to database calls.
- **C. Service Layer (Logic):**
    - Contains business logic and validation.
    - Depends on the Repository interface.
    - Accept context in methods and pass it down to the repository.
- **D. Handler Layer (Transport):**
    - Parse JSON -> Call Service -> Write JSON response.
    - **Task:** Extract context from the HTTP request (r.Context()) and pass it to the Service layer to enable timeout/cancellation propagation.
    - **Requirement:** Apply a **Context Deadline** (e.g., 5 seconds) using context.WithTimeout before passing it to the Service layer. This ensures operations abort if they exceed the time limit.

3. Final Wiring

- Wire all three layers - Repo -> Service -> Handler.
- **Validation:** Ensure data persists across server restarts.

Advanced:

- Use bulk POST api which accepts a CSV and creates products

Week 4

Upskilling Goal	1. Implementing Domain-Driven Design principles using Service and Repository patterns 2. Modeling relational data structures and managing entity dependencies (Foreign Keys) 3. Handling schema evolution, enforcing data integrity, and executing data migration strategies
Courses / Title	1. https://www.moesif.com/blog/technical/api-design/REST-API-Design-Filtering-Sorting-and-Pagination/ 2. https://www.mongodb.com/docs/manual/applications/data-models-relationships/ 3. (Advanced) https://gobyexample.com/waitgroups

Theme: Relational Data, Categories, and Migrations.

1. Category Domain (Days 1-2)

- **Model:** Create ProductCategory struct (ID, Title, Description).
  - *Examples:* "Food", "Kitchen Essentials".
- **Service & Repository:** Implement CategoryRepository and ProductCategoryService.
- **CRUD APIs:** Add endpoints to manage categories (Create, List, Update, Delete).

2. Product Relationships (Days 3-4)

- **Refactor Product:** Update Product model to include a CategoryID (reference to ProductCategory).
- **Filter API:** Fetch all products belonging to a specific category.

3. Enhanced Validation & Migration (Day 5)

- **New Rule:** Brand is now strictly required for all products.
- **Validation Logic:** Update ProductService to reject any creation/update without a valid Brand.
- **Data Migration (Handling Existing Data):**
  - How will we handle existing products?

Advanced:

- Use batching and wait groups to make use of the parallelism in go

Week 5

Upskilling Goal	1. Writing automated tests - unit tests and integration tests 2. Setting up development tools in your repository - coverage, testing scripts 3. Writing seed data - keeping repository developer friendly 4. Introduction to code reviews
Courses / Title	1. https://blog.jetbrains.com/go/2022/11/22/comprehensive-guide-to-testing-in-go/ 2. https://mortenvistisen.com/posts/integration-tests-with-docker-and-go 3. https://linearb.io/blog/code-review-on-github

Theme: Testing Strategies (Table-Driven) and CI/CD Practices.

1. Unit Testing (Days 1-2)

- **Strategy:** Use **Table-Driven Tests** (standard Golang style) to cover multiple scenarios (success, validation error, internal error) in a clean format.
- **Mocking:** Learn to mock the Repository interfaces (manually or using tools like mockery) to isolate Service logic.
- **Tasks:**
  - Write unit tests for ProductService.
  - Write unit tests for ProductCategoryService.
  - Ensure 100% coverage of business validation logic (e.g., negative prices, missing brands).

2. Integration Testing (Days 3-4)

- **Setup & Seeding:**
  - Write **Seed Scripts** to pre-populate the test DB with known categories and products.
  - Write an integration test that tests the various APIs created so far end-to-end with a local testing mongodb server

3. Workflow Evolution (Day 5)

- **Branching Strategy:**
  - **Stop merging directly to main.**
  - Transition to a Feature Branch workflow.
- **Pull Requests (PRs):**
  - Push the test cases from this week for **Mentor Review**.
  - **Peer Reviews:** Utilize fellow interns for initial reviews to prevent blocking on mentor availability.
  - **Merge Cycle:** Respond to feedback -> Refactor -> Merge only when approved.

Week 6

Upskilling Goal	1. Introduction to JS/HTML/CSS

	2. Consuming APIs via the frontend 3. Using debugging tools in the frontend
Courses / Title	1. https://blog.jetbrains.com/go/2022/11/22/comprehensive-guide-to-testing-in-go/ 2. https://mortenvistisen.com/posts/integration-tests-with-docker-and-go 3. https://linearb.io/blog/code-review-on-github

1. Create a new page with basic HTML components, and use plain HTML to create a display tile that displays Product information
2. Style the Product display tile with style sheets
3. Make API calls using vanilla javascript, and view incoming data in the browser console
4. Use developer tools in your browser to explore the DOM tree (HTML components), understand the box model, and view javascript queries

Advanced

1. Use the data from the API calls to modify the Product display tile dynamically
2. Taking it one step further, can we fetch data and render more HTML for the entire Product list?
3. Add some pure CSS animations

Week 7

Upskilling Goal	1. Introduction to typescript and react 2. Understanding the web development service 3. Debugging React applications and working with developer tools
Courses / Title	1. React Tutorial (with TS) - Start with Lab 4 2. Beginner tutorial for typescript 3. Your first component

1. Get familiar with typescript and React , including basic yarn commands to run a development server
2. Create basic components for displaying Product information, populating with dummy data

3. Add interactivity to a ProductList widget, such that clicking on a product element in the list expands to display product details (still using dummy data)

Advanced

1. Display the list of products using an <ProductList> and <Product> component
2. Add a header and navigation bar to the application

Week 8

Upskilling Goal	1. Consuming APIs, managing state, and handling data mutation on the frontend 2. Handling errors, loading state, and using higher-order components in React 3. Consuming routes and route parameters in a React application 4. Basic HTML-based browser navigation
Courses / Title	1. React.Dev | Managing state 2. Consuming APIs in React 3. Handling errors in React

1. Add functionality for a dedicated Product page, on which edits can be made to a product
2. Add interactivity for moving products across categories
3. Handle errors and display them to the user when some actions cannot be performed
4. Adding CRUD functionality over product categories is optional

(

Advanced

1. Add a product category page to view product categories, and clicking on a product category should take them to a product category page which displays product category information, and lists all products within that category - reuse <ProductList> or <Product> components here
2. Show loading spinners when refreshing data
3. Add further navigation - the product view page should display the category, which should link back to the category page

Week 9/10 (Optional)

NOTE *These 2 weeks are optional - wherein candidates may skip this week, and use it to catch up in case they have been lagging on their other weeks, or may choose to complete only parts of the 2 weeks.*

Upskilling Goal	1. Consuming APIs, managing state, and handling data mutation on the frontend 2. Handling errors, loading state, and using higher-order components in React 3. Consuming routes and route parameters in a React application 4. Basic HTML-based browser navigation 5. Introduction to FX 6. Introduction to GRPC And Twirp 7. Container technologies- Docker
Courses / Title	1. React.Dev | Managing state 2. Consuming APIs in React 3. Handling errors in React 4. https://uber-go.github.io/fx/get-started/index.html 5. https://connectrpc.com/docs/go/getting-started/

There are no advanced criteria for these 2 weeks, as these are optional weeks. The items in these 2 weeks are open-ended and serve as a guideline on what a product engineer's work looks like. Candidates choosing to complete this week should aim at innovation and research in terms of what they would like to build, and how they would like to implement a feature given to them. If you wish to build a different feature than the one outlined here, please consult with our mentor and figure out a path for the 2 weeks and set realistic goals.

1. End-to-end feature - reporting. Some ideas for reports:
    a. Add a feature for reporting that allows reports to be generated based on the number of products in each category. Allow filtering where categories within some product count ranges are ignored
    b. Users should be able to generate a report of the number of products across pre-segmented price ranges, for each category
    c. Users should be able to generate a list of products that are running low (quantity is below some threshold), or a list of product categories where more than 10% of the products are running low

2. Add navigation controls to your application - clicking on products in a report could navigate to a dedicated product, or open it within a sub-section on the page
3. Write a small approach note on how the implementation would look like - what are the new components you will introduce, and how they will work with the existing structure
4. Write an end-to-end playwright test for any single part of the application (the flow should ideally be exhaustive - for example, a test that tests that happy path on creating a product, editing, and viewing the edits)
5. Add support for exporting CSV reports
6. Change the wiring for the app created earlier to use FX
7. Learn GRPC and Twirp and update the API
8. Learn how and why of Docker and related containerization process

Track 3: Python backend with AI

Interneers Lab 2026

Track 3: Python backend with AI

Project Weekly Breakdown

Welcome to the Interneers Lab 2026! You would by now have an idea of the Interneers Lab, and what we are looking forward to for our next 10 weeks. By the end of these 10 weeks, we should have a very small, but end-to-end product for an "AI Powered Inventory Management System", and we will work towards building an intelligent IMS.

This document contains a weekly breakdown of what we should aim to achieve every week, and every week builds upon the work of the previous week. If you finish a week with time to spare and are looking for an additional challenge, feel free to attempt the "Advanced" set of goals as well for every week! If you are not able to complete your target for a week for any reason - work, or other commitments keeping you busy, don't worry! You can always catch up in the upcoming weeks, work with your mentors to figure out what is the best path for you to take and attempt the best as per what time permits.

Contents

Follow the content same as Track 1: Fullstack Python till Week 4

Week 1-4: Follow the content same as Track 1: Fullstack Python till Week 4

Week 5: Interactive Data Tools

ℹ️Note: This is a separate prototyping environment. Streamlit connects directly to MongoDB used for quick data exploration and AI feature demos.
Production systems would use proper API architecture from Weeks 1-4

Upskilling Goal	1. Introduction to **Streamlit** for rapid Python-based UI development. 2. Using **Jupyter Notebooks** as a playground for data exploration and AI prototyping. 3. Connecting your backend (Django/MongoEngine) to interactive front-end components.
Courses/Tutorials	• Streamlit Documentation: Main Concepts. • Building an Inventory App with Streamlit Fragments. • Jupyter Notebook Tutorial for Beginners. • Connecting Streamlit to MongoDB Atlas.

1. Create a `dashboard.py` file using Streamlit to display your current inventory in a table format.
2. Set up a Jupyter Notebook to test raw MongoEngine queries and visualize stock levels using `matplotlib` or `seaborn`.
3. Add basic Streamlit inputs (text boxes, buttons) to allow users to add or remove products directly from the UI.

Advanced

1. Implement a **Streamlit Sidebar** that allows filtering inventory by the "Product Category" logic built in Week 4.
2. Create a "Stock Alert" notification in Streamlit that turns red when an item's quantity falls below a threshold.

Week 6: Synthetic Data & Simulation

Upskilling Goal	1. Mastering **Prompt Engineering** to generate valid, structured JSON data. 2. Using **LLMs** to simulate realistic warehouse scenarios (e.g.,

	seasonal spikes, expiration dates). 3. Understanding **Pydantic** for validating AI-generated content against your database schema.
Courses/Tutorials	• Pydantic for LLMs: Schema, Validation & Prompts. • Using LLMs for Synthetic Data Generation. • OpenAI Guide: Structured Outputs.

1. Install openai library, set up API key - Write a script that asks: "Generate 5 product names for a toy store"
    a. Print the raw response and count tokens used
    b. Experiment: Change temperature from 0.0 to 1.5, observe differences -
    c. Calculate cost: What did this call cost?
2. Write a Python script in a Notebook that prompts an LLM to "Generate 50 products for a toy store" and returns them as a list of JSON objects matching your `Product` model.
3. Integrate **Pydantic** to validate that the LLM's output has the correct data types (e.g., `price` is a float, `quantity` is an int) before saving to MongoDB.
4. **Simulation:** Prompt the AI to generate a list of 10 "Future Stock Events" (e.g., "Expected delivery on Feb 1st") to test the audit trail logic from Week 4.

Advanced

1. Create a "Scenario Selector" in your Streamlit dashboard where users can choose a scenario (e.g., "Holiday Rush") and the AI populates the DB with relevant synthetic products and high stock levels

Week 7: Semantic Stock Search

Upskilling Goal	1. Understanding **Vector Embeddings** and how they capture semantic meaning. 2. Transitioning from keyword search to **Semantic Search** (similarity matching). 3. Implementing cosine similarity or vector database lookups in Python. 4. Building evaluation frameworks to measure search quality objectively 5. Understanding precision, recall, and relevance metrics for search systems
Courses/Tutorials	• Jay Alammar's article on word embeddings • Semantic Search in Python Step-by-Step.

	• Build a Semantic Search Engine in 5 Minutes. • Sentence-Transformers: Text to Vectors. • Introduction to t-SNE (to visualise embeddings in notebooks) • Precision and Recall for Ranking Systems

1. Install `sentence-transformers` and use a pre-trained model (like `all-MiniLM-L6-v2`) to convert your product descriptions into numerical vectors.
2. Manual Similarity Calculation
    a. Take 3 products: "Lego Castle", "Wooden Blocks", "Action Figure"
    b. Use sentence-transformers to get their embeddings - MANUALLY calculate cosine similarity between each pair using numpy
    c. Plot the 3 products in 2D using PCA (from 384D → 2D)
    d. Question: Why is Lego closer to Wooden Blocks than Action Figure?
3. Build a search function where a query for "construction toys" returns "Lego" even if the word "construction" isn't in the product name.
4. Build evalset and validate
    a. An evalset looks like this
    SEARCH_TEST_CASES = [{ "query": "construction toys", "relevant_products": ["lego_castle_001", "lego_city_002", "wooden_blocks_003"], "irrelevant_products": ["teddy_bear_010", "action_figure_015"] }, { "query": "gifts for toddlers", "relevant_products": ["soft_blocks_004", "plush_toy_005", "baby_rattle_006"], "irrelevant_products": ["puzzle_1000pc_020", "teen_board_game_025"] }....]
    b. A python script/function to ensure your search is performing well.
5. Update the Streamlit dashboard to include a "Semantic Search" bar alongside the traditional keyword search.

Advanced

1. Implement a "Find Similar Products" button next to each item in the UI that uses vector similarity to suggest related inventory items.
2. Comparing models
    a. all-MiniLM-L6-v2 (384 dimensions, fast)
    b. all-mpnet-base-v2 (768 dimensions, slower)
    c. Question: Does the larger model improve search quality for your inventory? Measure: Search for "toys for 5-year-olds" and manually rate top 5 results

Week 8: RAG

Upskilling Goal	1. Introduction to **Retrieval-Augmented Generation (RAG)**.

	2. Chunking documentation (PDFs, text files) to provide context to an LLM. 3. Building a "grounded" AI that only answers questions based on your specific inventory data. 4. Understand Langsmith tracing
Courses/Tutorials	• RAG Overview: Build an LLM Assistant with Streamlit. • Retrieval-Augmented Generation using LangChain. • Chunking Strategies for Better Retrieval

1. Upload a sample "Product Manual", "Return policy", "Vendor FAQ" as text files.
2. Use a Python library (like `langchain`) to split the text into chunks, embed them, and store them in a temporary vector store (MongoDB Vector search OR Chromadb is an option).
3. Retrieval: Build a function: retrieve_relevant_chunks(query, top_k=3)
    a. Test: "What's the return policy for damaged items?"
    b. Should return chunks from Return policy
    c. Eval suite for Retrieval
4. Build an "Ask the Expert" chat box in your Streamlit UI where users can ask questions like *"What is the warranty period for the Lego Castle?"*.
    a. Eval suite for RAG
5. Integrate with Langsmith and view the traces

Advanced

1. Combine RAG with your database: When a user asks about a product, retrieve both its documentation (RAG) and its current stock level (DB) to provide a complete answer.

Week 9&10: AI Quote Agent

Upskilling Goal	1. Implementing **AI Agents** that can perform multi-step reasoning. 2. Implementing function calling: Mapping natural language intent to Python functions 3. Managing tool-calling and agentic loops using frameworks like LangChain. 4. Policy enforcement: Combining LLM flexibility with deterministic business rules
Courses/Tutorials	• LangChain Agents Documentation - Focus on the ReAct pattern • OpenAI Function Calling Guide • Building an AI Agent with LangChain Tools.

	• Evaluating LLM Applications - Focus on testing strategies • LangChain Agents Documentation

1. Create three tools as Python functions that your agent can call:
    a. `get_product_info(product_id: str) -> dict`
    b. `check_inventory(product_id: str) -> dict`
    c. `calculate_quote(product_id: str, quantity: int) -> dict`
   Test each tool independently in a Jupyter notebook before connecting to the agent.
2. Define a "Quote Tool" in Python that accepts a product ID and quantity and calculates a price based on hard-coded discount rules (e.g., "10% off for quantities over 50").
3. Create an Agent that takes a natural language request (e.g., *"I need 60 building blocks for a school project, can I get a deal?"*). The Agent must:
    - Identify the product (Lego).
    - Check inventory levels (Week 3).
    - Call the "Quote Tool" to apply discounts.
    - Generate a final "Quote Invoice" as a JSON output or text summary in Streamlit.
4. Build eval suite for simple and complex scenarios

Advanced

1. Add a "Policy Check" step: If the LLM tries to give a discount larger than 20%, the Python logic must override it and provide a warning—ensuring the AI stays within "Success Boundaries".